TierMart Guide

A customer-tier store built with the State design pattern

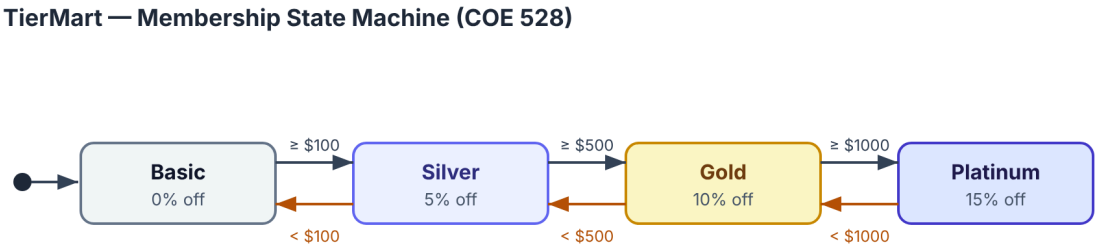
What this project is

TierMart is a small store application for COE 528 (Object-Oriented Analysis & Design). A customer's membership tier — **Basic → Silver → Gold → Platinum** — rises automatically as they spend and falls back on refunds, and each tier grants a bigger discount.

The store itself is simple on purpose. The project is really about the **design**:

- the **State design pattern** for the membership tiers (no `if/else` ladders anywhere),
- **design-by-contract** specifications on the methods that change state,
- a clean split between the **model**, the **business rules**, and two **front-ends** (a Swing GUI and a CLI) that share the exact same logic.

The State pattern — the heart of it



The membership state machine

Each tier is its own class implementing `MembershipState`, which knows just two things: the discount it grants, and — given a customer's lifetime spend — which tier the customer should now be in. The `Customer` is the **context**: it holds its current state and delegates.

When a purchase or refund changes a customer's lifetime spend, the customer asks its state to `review()` itself. The state checks the thresholds and, if needed, swaps in the neighbouring state. A single large purchase can skip tiers, because the customer re-reviews until the tier settles — while each state still only needs to know about its neighbours.

Why it's worth it: adding a fifth tier ("Diamond") later is *one new class* plus one threshold. No existing code changes. That's the open/closed principle in practice, and it's the difference between this and a `switch` statement you'd have to find and edit in five places.

The files

```
src/tiermart/
  model/
    Product.java            a catalogue item (price in cents, stock)
    Customer.java           THE CONTEXT: wallet, lifetime spend, current state
    Store.java              in-memory catalogue + customer list
    Receipt.java            result of a purchase (discount, tier before/after)
    membership/
      MembershipState.java  THE STATE interface: name(), discountRate(), review()
      Tiers.java            the thresholds, in one place
      BasicState / SilverState / GoldState / PlatinumState .java
  service/
    StoreService.java       THE BUSINESS RULES (purchase, refund) with contracts
    SampleData.java         seeds a store so the UIs have something to show
  app/
    Main.java               entry point: cli | gui | demo
    Cli.java                text front-end
    Gui.java                Swing desktop front-end
    SelfCheck.java          scripted proof the transitions are correct
```

The layering is deliberate — **model** (data + the state machine) knows nothing about the UI, **service** holds the rules, **app** is just front-ends calling the service. Swapping the in-memory store for a database would be a change in one layer.

Design by contract

The methods that change state carry a `@requires / @modifies / @effects` contract, and the preconditions are enforced at runtime (they throw rather than corrupt state). For example:

```
StoreService.purchase(customerId, productId, qty)
  @requires the customer and product exist && qty >= 1 && enough stock &&
            the customer can afford the discounted total
  @modifies the customer (wallet, lifetime spend, possibly tier) and the product (stock)
  @effects  applies the current-tier discount, charges the discounted total, reduces
            stock, re-evaluates the tier, and returns a Receipt
```

SOLID, concretely

- **Single responsibility** — model vs service vs UI; each state owns one tier.
- **Open/closed** — new tier = new class, nothing else changes.
- **Liskov** — any `MembershipState` works wherever the interface is expected.
- **Interface segregation** — `MembershipState` has exactly three methods.
- **Dependency inversion** — UIs depend on `StoreService`; `Customer` depends on the `MembershipState` interface, not the concrete tiers.

Running it, start to finish

1. Get a JDK (17+) and check it:

```
java -version && javac -version
```

If it's missing: `sudo apt install openjdk-17-jdk` (WSL/Ubuntu), `brew install openjdk@17` (macOS), or Temurin 17 from adoptium.net.

2. Build and run:

```
make build      # compile into out/
make demo       # the scripted State-pattern self-check
make run-cli    # interactive text menu
make run-gui    # Swing desktop window (needs a display; WSLg works)
```

3. What `make demo` prints — a customer driven up the tiers and back down:

```
action                    tier       discount  spent
-----------------------------------------------------------
start                     Basic      0%        $0.00
buy keyboard ($89)        Basic      0%        $89.00
buy monitor ($279)        Silver     5%        $368.00
buy monitor (5% off)      Gold       10%       $633.05
buy 2 monitors (10% off)  Platinum   15%       $1135.25
refund $900               Silver     5%        $235.25
All checks passed -- the State pattern promotes and demotes correctly.
```

In the GUI

`make run-gui` opens a window with a customer dropdown, a product dropdown, a quantity spinner, and Buy / Refund buttons. The customer table updates live — buy a few monitors for one customer and watch the tier and discount climb; hit "Refund $100" to see it walk back down. The GUI holds no business logic; every button just calls `StoreService` and redraws.

Putting it on GitHub

Create an empty `tiermart` repo on GitHub (no README), then:

```
cd TierMart
git init
git add .
git commit -m "COE 528: customer-tier store with the State design pattern"
git branch -M main
```

```
git remote add origin https://github.com/munevvercoskun/tiermart.git
git push -u origin main
```

`out/` (compiled classes) is git-ignored — source only.

What to say about it

The honest, strong version: *"It's a store where a customer's membership tier changes automatically with spending, and I built the tiers with the State design pattern — so the discount logic and the promotion rules live in the tier classes, not in if/else chains scattered through the code. Adding a new tier is one class. I put design-by-contract specs on the methods that change state and kept the business rules in one service layer that both the GUI and the CLI call."* That shows the OOAD concepts the course is actually about.